

**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Dokumentacja projektu

Wizja komputerowa

“Detekcja i klasyfikacja znaków drogowych”

Mateusz Pokrywka, Maciej Pereślucha
177143, 177139

Rzeszów, 18.6.2026

Spis treści

1	Opis problemu	3
2	Część teoretyczna	3
2.1	Algorytmy i filtry przetwarzania obrazu	3
2.1.1	Filtr Gaussa	3
2.1.2	Konwersja przestrzeni barw BGR→HSV i BGR→Grayscale . .	3
2.1.3	Progowanie kolorów i maska binarna	4
2.1.4	Morfologiczne filtry nieliniowe	4
2.1.5	Detekcja konturów	5
2.1.6	Filtry geometryczne konturów	5
2.1.7	Ekstrakcja ROI	5
2.2	Ekstrakcja cech – deskryptor HOG	6
2.2.1	Obliczanie gradientów	6
2.2.2	Histogram orientacji w komórce	6
2.2.3	Normalizacja bloków – L2-Hys	7
2.2.4	Wymiar wektora cech	7
2.3	Klasyfikacja – SVM	8
2.3.1	Idea i sformułowanie optymalizacyjne	8
2.3.2	Sformułowanie dualne i wektory nośne	9
2.3.3	Klasyfikacja wieloklasowa i estymacja prawdopodobieństwa . . .	10
2.3.4	Podział danych i metryki ewaluacyjne	10
3	Analiza danych	11
3.1	Dane do wytrenowania modelu SVM	11
3.2	Dane do testowania procesu detekcji i klasyfikacji	12
4	Pipeline procesu	13
5	Skrypt programu	14
6	Eksperymenty	17
6.1	Balansowanie zbioru	17
6.2	Dobór jądra SVM i parametru C	17

6.3	Parametry deskryptora HOG	18
6.4	Progi maski koloru i próg pewności	18
6.5	Wycięcie ROI: GTSRB kontra prawdziwa detekcja	18
6.6	Skuteczność całego systemu	19
7	Podsumowanie i wnioski	19

1 Opis problemu

Projekt dotyczy automatycznego wykrywania i rozpoznawania znaków drogowych na zdjęciach. System najpierw lokalizuje znak na obrazie (detekcja), a następnie przypisuje go do jednej z predefiniowanych kategorii (klasyfikacja). Do obu zadań wykorzystano klasyczne metody wizji komputerowej i klasyfikator SVM. Ze względu na realizację projektu w dwuosobowej grupie, w porozumieniu z prowadzącą ograniczono zakres do 5 klas: STOP, ustąp pierwszeństwa, zakaz wjazdu, droga z pierwszeństwem oraz ograniczenie prędkości do 30 km/h; zrezygnowano też z porównania z podejściami opartymi na głębokich sieciach neuronowych (YOLOv8-nano).

2 Część teoretyczna

Poniżej opisano wszystkie metody i algorytmy użyte w projekcie – od wstępnego przetwarzania obrazu, przez ekstrakcję cech, aż po klasyfikację. Każdy etap uzupełniono o konkretne parametry z implementacji.

2.1 Algorytmy i filtry przetwarzania obrazu

2.1.1 Filtr Gaussa

Filtr Gaussa rozmywa obraz, usuwając drobne szумы i zakłócenia. Matematycznie jest to spłot obrazu z dwuwymiarową funkcją Gaussa:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (2.1)$$

Obraz wygładzony: $I_g(x, y) = I(x, y) * G(x, y, \sigma)$. W implementacji użyto jądra 5×5 z automatycznym doбором σ przez OpenCV (`cv2.GaussianBlur(img, (5,5), 0)`). Rozmycie ogranicza ryzyko wykrycia fałszywych krawędzi na późniejszym etapie HOG.

2.1.2 Konwersja przestrzeni barw BGR→HSV i BGR→Grayscale

Przestrzeń HSV rozdziela kolor (Hue – odcień) od jasności (Value), co sprawia, że progowanie kolorów działa stabilnie przy różnym oświetleniu. Kanały wyznaczone są

ze składowych RGB:

$$V = \max(R, G, B) \quad (2.2)$$

$$S = \frac{V - \min(R, G, B)}{V} \quad (2.3)$$

$$H \in [0^\circ, 360^\circ) \quad (\text{wyznaczany z proporcji składowych RGB}) \quad (2.4)$$

W OpenCV kanał H jest skalowany do $[0, 180]$ (tj. $H_{cv} = H/2$), a $S, V \in [0, 255]$. Konwersja do skali szarości (potrzebna dla HOG) sprowadza obraz do jednego kanału intensywności:

$$I_{gray} = 0,114 B + 0,587 G + 0,299 R \quad (2.5)$$

2.1.3 Progowanie kolorów i maska binarna

Piksel trafia do maski, jeśli jego wartości (H_{cv}, S, V) mieszczą się w zadanym przedziale. Czerwień jest specyficzna – w przestrzeni HSV “zawija się” wokół 0° , więc trzeba sprawdzać dwa osobne zakresy kątowe:

$$H_{red} \in [0^\circ, 20^\circ) \cup [320^\circ, 360^\circ) \quad (2.6)$$

W skali OpenCV ($H_{cv} = H/2$) odpowiada to następującym progom z kodu:

$$M_{red1} : \quad H_{cv} \in [0, 10], S \in [30, 255], V \in [40, 255] \quad (2.7)$$

$$M_{red2} : \quad H_{cv} \in [160, 180], S \in [30, 255], V \in [40, 255] \quad (2.8)$$

$$M_{yellow} : \quad H_{cv} \in [20, 35], S \in [100, 255], V \in [100, 255] \quad (2.9)$$

Wszystkie trzy maski łączone są w jedną maskę kombinowaną:

$$M = M_{red1} \vee M_{red2} \vee M_{yellow} \quad (2.10)$$

2.1.4 Morfologiczne filtry nieliniowe

Operacje morfologiczne działają na czarno-białych maskach i pozwalają poprawić ich jakość – usunąć szum lub wypełnić dziury. Każda operacja korzysta z małego jądra B (np. 3×3 lub 7×7). Podstawowe operacje to erozja i dylatacja:

$$\text{Erozja:} \quad (A \ominus B)(p) = \min_{b \in B} A(p+b) \quad (2.11)$$

$$\text{Dylatacja:} \quad (A \oplus B)(p) = \max_{b \in B} A(p+b) \quad (2.12)$$

Łącząc je, otrzymujemy dwie użyteczne operacje złożone:

$$\text{Otwarcie: } A \circ B = (A \ominus B) \oplus B \quad (2.13)$$

$$\text{Zamknięcie: } A \bullet B = (A \oplus B) \ominus B \quad (2.14)$$

W implementacji stosowane są kolejno:

- 1) otwarcie z jądrem 3×3 – usuwa drobne szумы i pojedyncze piksele z tła maski,
- 2) zamknięcie z jądrem 7×7 – wypełnia dziury i luki wewnątrz wykrytych obiektów.

2.1.5 Detekcja konturów

Na oczyszczonej masce szukamy konturów, czyli ciągłych obrysów obiektów, korzystając z algorytmu Suzuki–Abe (`cv2.findContours`). Spośród wszystkich konturów $\{C_i\}$ wybieramy ten o największej powierzchni – zakładamy, że to nasz znak:

$$C^* = \arg \max_{C_i} A(C_i) \quad (2.15)$$

2.1.6 Filtry geometryczne konturów

Żeby odrzucić fałszywe detekcje (drzewa, budynki, pojazdy), każdy kandydat sprawdzany jest pod kątem kształtu. Używamy dwóch prostych miar:

$$\text{aspect ratio} = \frac{w}{h} \quad (2.16)$$

$$\text{solidity} = \frac{A_{\text{contour}}}{A_{\text{convex hull}}} \in (0, 1] \quad (2.17)$$

Znaki drogowe mają zbliżone proporcje ($\text{aspect ratio} \approx 1$) i wysoką zwartość ($\text{solidity} \approx 1$), bo ich kształt (koło, trójkąt, ośmiokąt) jest regularny. Obiekty odstające od tych wartości są odrzucane.

2.1.7 Ekstrakcja ROI

Po wybraniu konturu wycinamy z oryginalnego obrazu obszar zainteresowania (ROI – Region of Interest). Dodajemy marginesy $m = \lfloor 0,05 \cdot b_w \rfloor$ (5% szerokości), żeby nie

obcinać krawędzi znaku:

$$\text{ROI} = I[y_1 : y_2, x_1 : x_2], \quad \begin{cases} x_1 = \max(0, b_x - m) \\ y_1 = \max(0, b_y - m) \\ x_2 = \min(W, b_x + b_w + m) \\ y_2 = \min(H, b_y + b_h + m) \end{cases} \quad (2.18)$$

Jeśli segmentacja nie znajdzie żadnego konturu, ROI jest całym obrazem. Wycięty fragment skalowany jest do 64×64 px przed dalszym przetwarzaniem.

2.2 Ekstrakcja cech – deskryptor HOG

HOG (Histogram of Oriented Gradients) zamienia obraz na wektor liczb opisujący rozkład krawędzi. To dobry deskryptor dla znaków, bo znaki różnią się przede wszystkim kształtem, a HOG właśnie kształt koduje.

2.2.1 Obliczanie gradientów

Dla każdego piksela obrazu w skali szarości I wyznaczamy pochodne kierunkowe prostym operatorem różnicowym:

$$G_x(x, y) = I(x + 1, y) - I(x - 1, y) \quad (2.19)$$

$$G_y(x, y) = I(x, y + 1) - I(x, y - 1) \quad (2.20)$$

Następnie obliczamy magnitudę i kierunek gradientu:

$$m(x, y) = \sqrt{G_x^2(x, y) + G_y^2(x, y)} \quad (2.21)$$

$$\theta(x, y) = \arctan\left(\frac{G_y(x, y)}{G_x(x, y)}\right) \bmod 180^\circ \quad (2.22)$$

Kierunek jest liczony w zakresie $[0^\circ, 180^\circ)$ – bez znaku – bo krawędź pozioma wygląda tak samo z obu stron. To poprawia odporność na zmiany oświetlenia.

2.2.2 Histogram orientacji w komórce

Obraz dzielimy na siatkę małych komórek $p \times p$ pikseli (w implementacji $p = 8$). Dla każdej komórki budujemy histogram $b = 9$ przedziałów orientacyjnych, równomiernie pokrywających $[0^\circ, 180^\circ)$ (co 20° na przedział). Wartość k -tego przedziału:

$$h_k^{(c)} = \sum_{(x, y) \in c} m(x, y) \cdot \mathbf{1}\left[\theta(x, y) \in \left[\frac{180^\circ(k-1)}{b}, \frac{180^\circ k}{b}\right)\right], \quad k = 1, \dots, b \quad (2.23)$$

Gradient o dużej magnitudzie ma większy wkład do histogramu niż słaby – dlatego używamy $m(x, y)$ jako wagi.

2.2.3 Normalizacja bloków – L2-Hys

Żeby uniezależnić deskryptor od lokalnych zmian jasności, komórki grupujemy w bloki $c \times c$ (w implementacji $c = 2$, czyli 2×2 komórki). Histogramy wszystkich komórek bloku sklejamy w wektor $\mathbf{v} \in \mathbb{R}^{c^2b}$ i normalizujemy metodą L2-Hys w trzech krokach:

$$\mathbf{v}' = \frac{\mathbf{v}}{\sqrt{\|\mathbf{v}\|_2^2 + \varepsilon^2}} \quad (2.24)$$

$$v_i'' = \min(v_i', 0,2) \quad (\text{obcięcie}) \quad (2.25)$$

$$\mathbf{v}_{norm} = \frac{\mathbf{v}''}{\sqrt{\|\mathbf{v}''\|_2^2 + \varepsilon^2}} \quad (2.26)$$

Obcięcie wartości powyżej 0,2 (krok drugi) tłumi dominację silnych gradientów np. od obwódki znaku, dzięki czemu wnętrze znaku ma większy wpływ na opis.

2.2.4 Wymiar wektora cech

Tabela 2.1 zestawia parametry HOG z implementacji.

Parametr	Symbol	Wartość
Rozmiar obrazu	$W = H$	64 px
Rozmiar komórki	p	8 px
Komórek w bloku	c	2
Przedziałów orientacji	b	9

Tab. 2.1. Parametry deskryptora HOG użyte w implementacji.

Liczba bloków w każdym wymiarze (okno przesuwane co 1 komórkę):

$$n_{blocks} = \frac{W}{p} - c + 1 = \frac{64}{8} - 2 + 1 = 7 \quad (2.27)$$

Wymiar całego wektora cech:

$$d_{HOG} = \underbrace{n_{blocks}^2}_{\text{liczba bloków}} \cdot \underbrace{c^2 \cdot b}_{\text{cech na blok}} = 7^2 \cdot 2^2 \cdot 9 = 49 \cdot 36 = \mathbf{1764} \quad (2.28)$$

Każde zdjęcie w zbiorze uczącym reprezentowane jest jako wektor $\mathbf{x} \in \mathbb{R}^{1764}$.

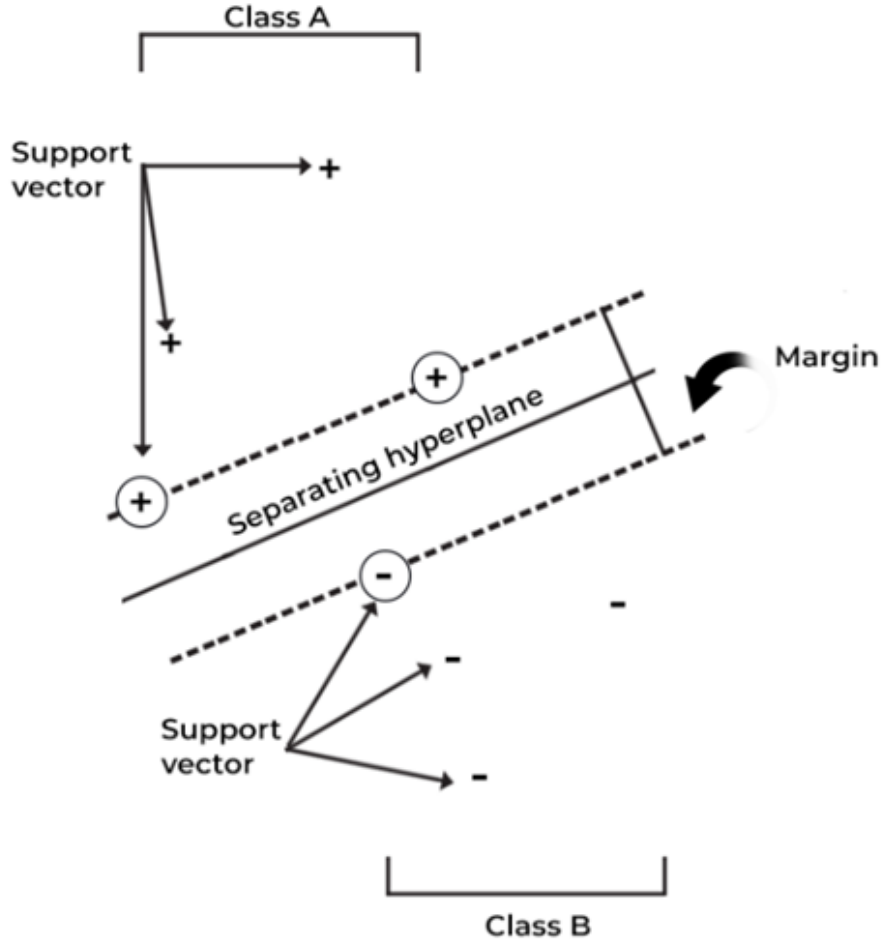
2.3 Klasyfikacja – SVM

2.3.1 Idea i sformułowanie optymalizacyjne

SVM (Support Vector Machine) to klasyfikator, który szuka hiperpłaszczyzny decyzyjnej $\mathbf{w}^T \mathbf{x} + b = 0$ maksymalnie oddzielającej klasy od siebie (patrz Rys. 2.1). Im szerszy margines między klasami, tym lepsza generalizacja na nowych danych. W wersji soft-margin, która dopuszcza pewne błędy klasyfikacji, zadanie optymalizacji wygląda następująco:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{przy} \quad y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2.29)$$

Parametr C steruje kompromisem – duże C oznacza, że błędy są mocno karane (wąski margines, ryzyko overfittingu), małe C pozwala na więcej błędów treningowych w zamian za szerszy margines. W implementacji użyto jądra liniowego (`kernel='linear'`) z domyślną wartością $C = 1,0$.



Rys. 2.1. Hiperpłaszczyzna decyzyjna SVM z zaznaczonym marginesem i wektorami nośnymi.

2.3.2 Sformułowanie dualne i wektory nośne

Równoważne, dualnie sformułowanie problemu pozwala wyrazić rozwiązanie przez punkty treningowe przy użyciu mnożników Lagrange'a $\alpha_i \geq 0$:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad \text{przy} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \quad (2.30)$$

Punkty, dla których $\alpha_i > 0$ nazywamy wektorami nośnymi (support vectors) – to one wyznaczają hiperpłaszczyznę, reszta punktów treningowych nie ma znaczenia. Funkcja decyzyjna:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i: \alpha_i > 0} \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b \right) \quad (2.31)$$

2.3.3 Klasyfikacja wieloklasowa i estymacja prawdopodobieństwa

SVM w podstawowej wersji rozwiązuje problem dwuklasowy. Scikit-learn rozszerza go na wiele klas strategią jeden-na-jeden (OvO – One vs One): dla każdej pary klas trenowany jest osobny klasyfikator. Dla $K = 5$ klas daje to:

$$\binom{K}{2} = \binom{5}{2} = 10 \quad \text{klasyfikatorów binarnych} \quad (2.32)$$

Wynikowa klasa wyłaniana jest głosowaniem – wygrywa ta, która wygrała najwięcej pojedynków. Flaga `probability=True` w kodzie włącza skalowanie Platt’a, które zamienia surowe wyniki klasyfikatorów na prawdopodobieństwa:

$$P(y = k | \mathbf{x}) = \frac{1}{1 + e^{A_k f_k(\mathbf{x}) + B_k}} \quad (2.33)$$

Parametry A_k i B_k dopasowywane są automatycznie. Wartości te widoczne są w kodzie jako `confidence` – procent pewności klasyfikatora.

2.3.4 Podział danych i metryki ewaluacyjne

Zbalansowany zbiór 3900 próbek dzielony jest losowo w proporcji 80/20 (`random_state=42` gwarantuje powtarzalność):

$$|\mathcal{D}_{train}| = 0,8 \cdot 3900 = 3120 \quad |\mathcal{D}_{test}| = 0,2 \cdot 3900 = 780 \quad (2.34)$$

Do oceny modelu używamy czterech standardowych metryk, wyznaczanych osobno dla każdej klasy k , gdzie TP_k , FP_k , FN_k to odpowiednio: prawdziwie pozytywne, fałszywie pozytywne i fałszywie negatywne predykcje dla klasy k :

$$\text{Accuracy} = \frac{\sum_k TP_k}{n} \quad (2.35)$$

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k} \quad (2.36)$$

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \quad (2.37)$$

$$F_{1,k} = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (2.38)$$

3 Analiza danych

3.1 Dane do wytrenowania modelu SVM

Do treningu użyto zbioru GTSRB (German Traffic Sign Recognition Benchmark) – jednego z najpopularniejszych zbiorów do rozpoznawania znaków drogowych, zawierającego ponad 50 000 zdjęć w 43 klasach. Korzystaliśmy wyłącznie z części treningowej, ponieważ zdjęcia są tam posortowane według klas i mają dostępne etykiety. Obrazy mają różne wymiary i zapisane są w formacie PNG – przeważającą część kadru zajmuje sam znak (patrz Rys. 3.1).



Rys. 3.1. Przykładowe zdjęcie z bazy GTSRB (klasa 0 – ograniczenie prędkości do 20 km/h).

Spośród 43 klas wybraliśmy 5 odpowiadających zakresowi projektu. Liczba zdjęć w tych klasach była jednak bardzo nierówna (Tabela 3.1) – od 780 (STOP) do 2220

(ograniczenie 30).

Klasa	Liczba zdjęć
Ograniczenie prędkości 30 km/h	2220
Droga z pierwszeństwem	2100
Ustąp pierwszeństwa	2160
Zakaz wjazdu	1110
STOP	780
Łącznie	8370

Tab. 3.1. Pierwotna (niezbalansowana) liczebność wybranych klas w bazie GTSRB.

Taka nierównowaga sprawia, że model uczy się głównie klas licznych, a słabiej rozpoznaje te rzadsze. Dlatego zbiór **zbalansowaliśmy**: z każdej klasy wylosowaliśmy tyle samo zdjęć, ile liczy klasa najmniejsza (STOP – 780). Po zbalansowaniu każda z 5 klas ma 780 zdjęć, co daje $5 \times 780 = 3900$ zdjęć uczących. Eksperyment w rozdziale 6 potwierdza, że balansowanie poprawia rozpoznawanie rzadszych klas.

Zbalansowany zbiór podzielono losowo na treningowy (80%) i testowy (20%) funkcją `train_test_split()` z biblioteki `scikit-learn` [5] (`random_state=42` – por. równanie 2.34).

3.2 Dane do testowania procesu detekcji i klasyfikacji

Do oceny całego potoku (nie tylko modelu SVM) użyliśmy 25 zdjęć (po 5 na każdą klasę) z prawdziwych dróg, pobranych z internetu. W odróżnieniu od GTSRB znaki zajmują na nich mniejszą część kadru, a tło jest zróżnicowane – są więc bliższe warunkom rzeczywistym (Rys. 3.2).



Rys. 3.2. Przykładowe zdjęcia testowe do oceny skuteczności systemu.

4 Pipeline procesu

Cały proces realizowany jest jako sekwencja pięciu etapów – każdy etap przetwarza wynik poprzedniego:

- 1) **Wstępne przetwarzanie:** wczytanie obrazu, zastosowanie filtru Gaussa 5×5 (równanie 2.1), konwersja BGR→HSV.
- 2) **Segmentacja i binaryzacja:** progowanie trzech zakresów kolorów (równania 2.7–2.9) i połączenie masek (równanie 2.10); następnie otwarcie jądrem 3×3 i zamknięcie jądrem 7×7 (równania 2.13–2.14).
- 3) **Detekcja i filtracja geometryczna:** wyznaczenie konturów, wybór największego (równanie 2.15), weryfikacja aspect ratio i solidity (równania 2.16–2.17).
- 4) **Ekstrakcja ROI i cech:** wycięcie obszaru zainteresowania z marginesem 5% (równanie 2.18), skalowanie do 64×64 px, konwersja do skali szarości, ekstrakcja wektora HOG $\in \mathbb{R}^{1764}$ (równanie 2.28).

- 5) **Klasyfikacja SVM:** przekazanie wektora cech do wytrenowanego klasyfikatora liniowego SVM (równanie 2.31), przypisanie do jednej z 5 klas wraz z prawdopodobieństwem (równanie 2.33).

5 Skrypt programu

Listing 5.1. Pełny potok treningu: preprocessing HSV + ekstrakcja HOG + SVM

```
1 import os
2 import cv2
3 import numpy as np
4 from skimage.feature import hog
5 from sklearn.model_selection import train_test_split
6 from sklearn.svm import SVC
7 from sklearn.metrics import (accuracy_score, classification_report,
8                               confusion_matrix, ConfusionMatrixDisplay)
9 import matplotlib.pyplot as plt
10 import joblib
11
12 DATA_DIR = '../data/db_balanced' # zbalansowany zbior (780
13     zdjec/klase)
14 MODEL_SAVE_PATH = '../models/svm_model.pkl'
15 IMG_SIZE = (64, 64)
16
17 X, labels, rois_for_display = [], [], []
18 os.makedirs(os.path.dirname(MODEL_SAVE_PATH), exist_ok=True)
19
20 for class_name in os.listdir(DATA_DIR):
21     class_path = os.path.join(DATA_DIR, class_name)
22     if not os.path.isdir(class_path):
23         continue
24     for img_name in os.listdir(class_path):
25         img_path = os.path.join(class_path, img_name)
26         img = cv2.imread(img_path)
27         if img is None or img.size == 0:
28             print(f"[POMINIETO] Uszkodzony plik: {img_path}")
29             continue
30         try:
31             # --- Etap 1: Odszumianie i maskowanie HSV ---
32             img_blurred = cv2.GaussianBlur(img, (5, 5), 0)
33             hsv = cv2.cvtColor(img_blurred, cv2.COLOR_BGR2HSV)
34
35             # Czerwien w HSV dzieli sie na dwa zakresy katowe
36             lower_red1, upper_red1 = np.array([0,30,40]), np.
37             array([10,255,255])
38             lower_red2, upper_red2 = np.array([160,30,40]), np.
39             array([180,255,255])
40             lower_yellow, upper_yellow = np.array([20,100,100]),np.
41             array([35,255,255])
42
43             mask = (cv2.inRange(hsv, lower_red1, upper_red1) +
44                     cv2.inRange(hsv, lower_red2, upper_red2) +
45                     cv2.inRange(hsv, lower_yellow, upper_yellow))
```

```

43         # OPEN 3x3 usuwa szum, CLOSE 7x7 wypelnia dziury w masce
44         kernel_open = np.ones((3, 3), np.uint8)
45         kernel_close = np.ones((7, 7), np.uint8)
46         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_open
47     )
48         mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
49     kernel_close)
50
51         contours, _ = cv2.findContours(
52             mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
53
54         if contours:
55             largest = max(contours, key=cv2.contourArea)
56             bx, by, bw, bh = cv2.boundingRect(largest)
57             margin = int(0.05 * bw)
58             x1 = max(0, bx - margin)
59             y1 = max(0, by - margin)
60             x2 = min(img.shape[1], bx + bw + margin)
61             y2 = min(img.shape[0], by + bh + margin)
62             roi = img[y1:y2, x1:x2]
63             if roi.size == 0:
64                 roi = img.copy()
65             else:
66                 roi = img.copy()
67
68         # --- Etap 2: Resize -> Grayscale -> HOG ---
69         # Parametry HOG daja wektor 1764 cech (patrz dokumentacja)
70         img_resized = cv2.resize(roi, IMG_SIZE)
71         gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
72         features = hog(gray, orientations=9, pixels_per_cell=(8,
73     8),
74     cells_per_block=(2, 2), block_norm='L2-Hys',
75     visualize=False)
76
77         X.append(features)
78         labels.append(class_name)
79         rois_for_display.append(cv2.cvtColor(roi, cv2.
80     COLOR_BGR2RGB))
81
82     except Exception as e:
83         print(f"[BLAD] {img_path}: {e}")
84         continue
85
86     X = np.array(X)
87     y = np.array(labels)
88     rois_for_display = np.array(rois_for_display, dtype=object)
89
90     print(f"[INFO] Przetworzono {len(X)} zdj. z {len(np.unique(y))} klas."
91 )
92
93     # --- Trening ---
94     X_train, X_test, y_train, y_test, rois_train, rois_test =
95     train_test_split(
96         X, y, rois_for_display, test_size=0.2, random_state=42)
97
98     svm_model = SVC(kernel='linear', probability=True)
99     svm_model.fit(X_train, y_train)

```



```

94
95 # --- Ewaluacja ---
96 y_pred = svm_model.predict(X_test)
97 accuracy = accuracy_score(y_test, y_pred)
98 print(f"\n[WYNIK] Accuracy: {accuracy * 100:.2f}%\n")
99 print(classification_report(y_test, y_pred))
100
101 # --- Zapis ---
102 joblib.dump(svm_model, MODEL_SAVE_PATH)
103 print(f"[SUKCES] Model zapisany: {MODEL_SAVE_PATH}")
104
105 # --- Wizualizacja 1: Macierz pomylek ---
106 cm = confusion_matrix(y_test, y_pred, labels=svm_model.classes_)
107 disp = ConfusionMatrixDisplay(confusion_matrix=cm,
108                               display_labels=svm_model.classes_)
109 fig, ax = plt.subplots(figsize=(10, 8))
110 disp.plot(cmap='Blues', ax=ax, xticks_rotation=45, values_format='d')
111 plt.title('Macierz Pomylek -- SVM (HOG + OpenCV)', fontsize=15,
112          fontweight='bold')
113 plt.tight_layout()
114 plt.show()
115
116 # --- Wizualizacja 2: Galeria predykcji (po 1 przykladzie z klasy) ---
117 unique_classes = np.unique(y_test)
118 num_classes = len(unique_classes)
119 fig2, axes2 = plt.subplots(1, num_classes, figsize=(18, 4))
120 fig2.suptitle('Predykcje na zbiorze testowym', fontsize=16,
121              fontweight='bold', y=1.1)
122
123 if num_classes == 1: # axes2 nie jest iterowalny dla 1 klasy
124     axes2 = [axes2]
125
126 for i, cls in enumerate(unique_classes):
127     idx = np.where(y_test == cls)[0][0]
128     feature_vector = X_test[idx].reshape(1, -1)
129     true_label = y_test[idx]
130     display_img = rois_test[idx]
131
132     prediction = svm_model.predict(feature_vector)[0]
133     confidence = np.max(svm_model.predict_proba(feature_vector)) * 100
134
135     title_color = 'green' if prediction == true_label else 'red'
136     axes2[i].imshow(display_img)
137     axes2[i].set_title(
138         f"Predykcja: {prediction}\nPewnosc: {confidence:.1f}%\n"
139         f"Prawda: {true_label}",
140         fontsize=11, color=title_color, fontweight='bold')
141     axes2[i].axis('off')
142
143 plt.tight_layout()
144 plt.show()

```

6 Eksperymenty

Eksperymenty miały dwa cele: dobrać parametry potoku i sprawdzić skuteczność systemu. Model oceniamy na dwa sposoby: (1) na zbiorze testowym GTSRB (20% danych) oraz (2) na zdjęciach z prawdziwych dróg, które pokazują działanie całego potoku detekcji. Wszystkie eksperymenty wykonano przy stałym ziarnie losowym (`random_state=42`).

6.1 Balansowanie zbioru

Najpierw sprawdziliśmy wpływ zbalansowania klas. Model trenowany na zbiorze niezbalansowanym osiąga 89,84% trafności, a na zbalansowanym 90,77%. Co ważniejsze, na macierzy pomyłek wyraźnie poprawia się rozpoznawanie klas wcześniej mniej licznych. Dlatego w finalnym systemie używamy zbioru zbalansowanego.

6.2 Dobór jądra SVM i parametru C

Porównaliśmy jądro liniowe i RBF dla kilku wartości parametru C (Tabela 6.1). Najwyższą trafność na GTSRB daje jądro RBF z $C = 10$ (95,5%). Mimo to w finalnym systemie zostawiliśmy jądro **liniowe**: jak pokazał eksperyment z ROI (podrozdział 6.5), wyższy wynik na GTSRB nie przekłada się na lepszą detekcję na realnych zdjęciach, a model liniowy daje stabilniejsze, lepiej skalibrowane prawdopodobieństwa.

Jądro	C	Skalowanie cech	Accuracy [%]
RBF	10	nie	95,52
RBF	1	tak	94,56
liniowe	0,1	nie	92,83
liniowe	1	nie	89,84
liniowe	1	tak	89,25

Tab. 6.1. Wybrane wyniki porównania konfiguracji SVM (zbiór testowy GTSRB).

Sprawdziliśmy też skalowanie cech (`StandardScaler`) oraz wyważenie klas (`class_weight`) – żadne z nich nie poprawiło wyniku, ponieważ deskryptor HOG jest już znormalizowany metodą L2-Hys.

6.3 Parametry deskryptora HOG

Porównaliśmy różne rozmiary komórki, liczby orientacji i wielkości bloku (Tabela 6.2). Co ciekawe, większy wektor cech nie oznacza wyższej trafności – najkrótszy wektor (komórka 16×16) wypada nawet najlepiej. Zostawiliśmy konfigurację 8×8 px, 9 orientacji i blok 2×2 (wektor 1764 cech) jako sprawdzony kompromis między dokładnością a długością wektora.

Komórka [px]	Orientacje	Blok	Accuracy [%]
8	6	2	90,80
8	9	2	89,84
8	12	2	89,84
16	9	2	92,77
8	9	3	91,22

Tab. 6.2. Wpływ parametrów HOG na trafność (zbiór testowy GTSRB).

6.4 Progi maski koloru i próg pewności

W treningu maska koloru używa niższych progów nasycenia i jasności ($S, V \geq 30, 40$), ponieważ na wykadrowanych zdjęciach GTSRB tło prawie nie istnieje. W detekcji na zdjęciach drogowych podnosimy je do 70,50 – niższe progi przepuszczają zbyt wiele fałszywych obszarów o zbliżonym kolorze. Podobnie próg pewności klasyfikatora (0,70–0,85) jest kompromisem: zbyt niski wpuszcza błędne etykiety na tle, a zbyt wysoki pomija trudniejsze znaki.

6.5 Wycięcie ROI: GTSRB kontra prawdziwa detekcja

To najważniejszy eksperyment. Sprawdziliśmy, czy klasyfikator powinien uczyć się na wyciętym ROI, czy na całym obrazie. Na zbiorze GTSRB wynik jest zaskakujący: model uczony *bez* ROI osiąga prawie 100%, a z ROI około 90%. GTSRB to gotowe wycięcia znaku, więc dodatkowe wycinanie tylko psuje obraz.

Kluczowa jest jednak ocena na **prawdziwych zdjęciach** (Tabela 6.3). Tu sytuacja się odwraca: model uczony z ROI poprawnie rozpoznaje 72% znaków, a bez ROI tylko 64%. Powód: w detekcji klasyfikator zawsze dostaje wycinek znaku z całego

zdjęcia, więc musi być uczony na takich samych wycinkach. Wyższy wynik na GTSRB nie oznacza zatem lepszego systemu – liczy się zgodność sposobu przetwarzania obrazu w treningu i w detekcji.

Wariant uczenia	Trafność na GTSRB	Skuteczność detekcji
z ROI (finalny)	~90%	72%
bez ROI	~100%	64%

Tab. 6.3. Lepszy wynik na GTSRB (bez ROI) nie przekłada się na lepszą detekcję na realnych zdjęciach.

6.6 Skuteczność całego systemu

Finalny system (uczenie z ROI, zbiór zbalansowany, jądro liniowe) przetestowaliśmy na 25 zdjęciach z prawdziwych dróg (po 5 na każdą klasę). System wykrył znak na 20 z 25 zdjęć (80%), a 18 z 25 sklasyfikował w pełni poprawnie (72%). Najlepiej rozpoznawane są znaki zakazu wjazdu i STOP; najwięcej pominieć dotyczy ograniczenia 30 – wynika to z różnic między niemieckimi znakami z bazy GTSRB a znakami na zdjęciach testowych.

7 Podsumowanie i wnioski

Co zadziało. Zbudowany system łączy klasyczny potok przetwarzania obrazu (maska koloru HSV, morfologia, kontury, wycięcie ROI) z deskryptorem HOG i klasyfikatorem SVM. Na zbiorze testowym GTSRB osiąga około 90% trafności, a na prawdziwych zdjęciach drogowych poprawnie rozpoznaje 72% znaków. Najlepiej sprawdza się dla znaków wyraźnych i widzianych od przodu.

Wnioski z eksperymentów. Najważniejsza obserwacja: *wyższy wynik na zbiorze testowym nie zawsze oznacza lepszy system*. Zarówno uczenie bez ROI, jak i jądro RBF dawały wyższą trafność na GTSRB, ale gorzej (lub mniej stabilnie) działały na realnej detekcji. Dlatego trening i detekcja muszą stosować ten sam sposób przetwarzania obrazu. Zbalansowanie zbioru poprawiło rozpoznawanie rzadszych klas, a skalowanie cech i wyważanie klas okazały się zbędne, bo HOG jest już znormalizowany.

Ograniczenia. System ma trzy główne słabości:

- **Różnica dziedzin** – model uczył się na niemieckich znakach GTSRB, więc na zdjęciach z innych krajów (inny font, kąt, oświetlenie) myli się częściej, zwłaszcza przy ograniczeniu 30.
- **Brak klasy „tło”** – SVM zawsze przypisuje jedną z 5 klas, więc czerwone lub żółte fragmenty tła (cegła, dachówki) bywają błędnie oznaczane jako znaki.
- **Wrażliwość maski koloru** – wyblakłe lub zacienione znaki bywają pomijane, bo nie mieszczą się w progach maski.

Możliwe ulepszenia. W przyszłości warto: dodać klasę „tło” z przykładami negatywnymi, aby ograniczyć fałszywe detekcje; rozszerzyć zbiór uczący o znaki z różnych krajów lub zastosować augmentację danych; a także dodać tłumienie nakładających się detekcji i obsługę wielu znaków na jednym zdjęciu.

Literatura

- [1] J. Marnik: *Materiały wykładowe z przedmiotu "Wizja komputerowa"*, Politechnika Rzeszowska 2025/26.
- [2] J. Marnik: *Propozycja zadań projektowych z przedmiotu "Wizja komputerowa"*, Politechnika Rzeszowska 2025/26.
- [3] J. Marnik: *Opis teoretyczne do ćwiczeń z przedmiotu "Wizja komputerowa"*, Politechnika Rzeszowska 2025/26.
- [4] M. Pokrywka: *Analiza wpływu hiperparametrów maszyny wektorów nośnych przy pomocy zbioru Haberman*, Github/MATP0K 2024.
- [5] F. Pedregosa et al.: *Scikit-learn: Machine Learning in Python*, JMLR 2011